Task 4: Real-Time Communication App

Express + Socket.io + WebRTC + Web Crypto. Multi-user video, screen sharing, file sharing, whiteboard, auth, end-to-end encrypted chat/files/whiteboard (AES-GCM).

GitHub repository name: **CodeAlpha_RealTimeCommunicationApp**

How to run

```
cd CodeAlpha_RealTimeCommunicationApp
npm install
npm start        # open http://localhost:3000  (Node 18+)
```

Overview

Features: multi-user video (WebRTC mesh), screen sharing, file sharing (5 MB max), whiteboard, user auth, end-to-end encrypted chat/files/whiteboard (AES-GCM key derived from the room password). Needs localhost or HTTPS for camera. Test with two tabs, same room and password. No TURN server, so some networks will not connect.

Emoji in the UI are omitted from this listing only (the PDF font cannot show them).

CodeAlpha_RealTimeCommunicationApp/package.json

```
{ "name": "codealpha-rtc", "version": "1.0.0", "main": "server.js", "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2", "better-sqlite3": "*", "bcryptjs": "^2.4.3", "jsonwebtoken": "^9.0.2",
      "socket.io": "^4.7.5" } }
```

CodeAlpha_RealTimeCommunicationApp/auth.js

```
const jwt = require('jsonwebtoken'), bcrypt = require('bcryptjs');
const SECRET = process.env.JWT_SECRET || 'dev-secret-change-me';
const sign = u => jwt.sign({ id: u.id, username: u.username }, SECRET, { expiresIn: '7d' });
const verify = t => jwt.verify(t, SECRET);
const auth = (req, res, next) => {
  try { req.user = verify((req.headers.authorization || '').replace('Bearer ', '')); next(); }
  catch { res.status(401).json({ error: 'Login required' }); }
};
// Registers POST /api/register and /api/login (needs a `users(id, username, password)` table)
function authRoutes(app, db) {
  app.post('/api/register', (req, res) => {
    const { username, password } = req.body || {};
    if (!/^\w{3,20}$/.test(username || '') || (password || '').length < 6)
      return res.status(400).json({ error: 'Username 3-20 chars (letters/digits/_), password 6+ chars' });
    try {
      const id = db.prepare('INSERT INTO users(username,password) VALUES(?,?)').run(username,
          bcrypt.hashSync(password, 10)).lastInsertRowid;
      res.json({ token: sign({ id, username }), username });
    } catch { res.status(409).json({ error: 'Username already taken' }); }
  });
  app.post('/api/login', (req, res) => {
    const { username, password } = req.body || {};
    const u = db.prepare('SELECT * FROM users WHERE username=?').get(username || '');
    if (!u || !bcrypt.compareSync(password || '', u.password)) return res.status(401).json({ error: 'Wrong username
        or password' });
    res.json({ token: sign(u), username: u.username });
  });
}
module.exports = { sign, verify, auth, authRoutes };
```

CodeAlpha_RealTimeCommunicationApp/server.js

```
const express = require('express'), http = require('http'), { Server } = require('socket.io'), Database =
    require('better-sqlite3');
const { authRoutes, verify } = require('./auth');
const db = new Database('rtc.db');
db.exec('CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE, password TEXT)');
const app = express(); app.use(express.json()); app.use(express.static('public')); authRoutes(app, db);
const server = http.createServer(app);
const io = new Server(server, { maxHttpBufferSize: 2e7 });
io.use((s, next) => { try { s.user = verify(s.handshake.auth.token); next(); } catch { next(new Error('auth')); } });
    // authenticated sockets only

io.on('connection', s => {
  s.on('join', room => {
    if (typeof room !== 'string' || !room || s.room) return;
    s.room = room;
    const peers = [...(io.sockets.adapter.rooms.get(room) || [])].map(id => ({ id, username:
        io.sockets.sockets.get(id).user.username }));
```

```
    s.join(room); s.emit('peers', peers);
    s.to(room).emit('peer-joined', { id: s.id, username: s.user.username });
  });
  s.on('signal', ({ to, data }) => { if (s.room && io.sockets.sockets.get(to)?.room === s.room)
      io.to(to).emit('signal', { from: s.id, data }); }); // WebRTC signalling
  // The server only relays opaque ciphertext (end-to-end encrypted in the browser)
  ['chat', 'file', 'draw', 'clear'].forEach(ev => s.on(ev, p => s.room && s.to(s.room).emit(ev, { from:
      s.user.username, p })));
  s.on('disconnect', () => s.room && s.to(s.room).emit('peer-left', s.id));
});
server.listen(process.env.PORT || 3000, () => console.log('RTC app on http://localhost:3000'));
```

CodeAlpha_RealTimeCommunicationApp/public/index.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport"
    content="width=device-width,initial-scale=1"><title>MeetAlpha</title><link rel="stylesheet" href="style.css">
<style>.meet{display:grid;grid-template-columns:2fr
    1fr;gap:12px}@media(max-width:800px){.meet{grid-template-columns:1fr}}
.tile{position:relative;background:#000;border-radius:8px;overflow:hidden}.tile video{width:100%;display:block}.tile
    span{position:absolute;left:6px;bottom:6px;background:#0009;color:#fff;padding:2px
    6px;border-radius:4px;font-size:12px}
canvas{width:100%;background:#fff;border-radius:8px;touch-action:none;border:1px solid
    #cbd5e1}#log{height:360px;overflow:auto;background:#fff;border-radius:8px;padding:8px;margin-bottom:6px}.bar{margi
    n:8px 0}.bar button,.bar .btn{margin-right:6px}</style></head>
<body><header><b> MeetAlpha</b><span id="nav"></span></header><main id="app"></main>
<script src="common.js"></script><script src="/socket.io/socket.io.js"></script><script
    src="app.js"></script></body></html>
```

CodeAlpha_RealTimeCommunicationApp/public/app.js

```
const ICE = { iceServers: [{ urls: 'stun:stun.l.google.com:19302' }] }; // add a TURN server for strict NATs
let socket, key, cam, ctx, peers = {}, names = {}, sharing = false, drawing = false, last, chain = Promise.resolve();

if (!token) $('#app').innerHTML = authBox();
else {
  $('#nav').innerHTML = `${esc(me)} <a onclick="logout()">Logout</a>`;
  $('#app').innerHTML = `<div class="card narrow"><h3>Join a room</h3><input id="room" placeholder="Room name"><input
      id="pw" type="password" placeholder="Room password (encryption key)"><button
      onclick="join()">Join</button><p><small>Everyone in the room must use the same password. It never leaves your
      browser.</small></p></div>`;
}

// ---- End-to-end encryption: AES-GCM key derived from the room password (PBKDF2) ----
async function deriveKey(pw, room) {
  const base = await crypto.subtle.importKey('raw', new TextEncoder().encode(pw), 'PBKDF2', false, ['deriveKey']);
  return crypto.subtle.deriveKey({ name: 'PBKDF2', salt: new TextEncoder().encode('meetalpha:' + room), iterations:
      150000, hash: 'SHA-256' }, base, { name: 'AES-GCM', length: 256 }, false, ['encrypt', 'decrypt']);
}
function toB64(buf) { const a = new Uint8Array(buf); let s = ''; for (let i = 0; i < a.length; i += 0x8000) s +=
    String.fromCharCode.apply(null, a.subarray(i, i + 0x8000)); return btoa(s); }
const fromB64 = s => Uint8Array.from(atob(s), c => c.charCodeAt(0));
async function enc(o) { const iv = crypto.getRandomValues(new Uint8Array(12)); const ct = await
    crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, new TextEncoder().encode(JSON.stringify(o))); return
    toB64(iv) + '.' + toB64(ct); }
async function dec(s) { const [i, c] = s.split('.'); return JSON.parse(new TextDecoder().decode(await
    crypto.subtle.decrypt({ name: 'AES-GCM', iv: fromB64(i) }, key, fromB64(c)))); }

function log(t, raw) { const d = document.createElement('div'); d.innerHTML = raw ? t : esc(t); $('#log').append(d);
    $('#log').scrollTop = 1e9; }

async function join() {
  const room = $('#room').value.trim(), pw = $('#pw').value;
  if (!room || !pw) return alert('Room and password required');
  key = await deriveKey(pw, room);
  try { cam = await navigator.mediaDevices.getUserMedia({ video: true, audio: true }); }
  catch { alert('Camera/mic unavailable (needs https or localhost + permission). Joining without media.'); cam = new
      MediaStream(); }
  $('#app').innerHTML = `<div class="meet"><div><div id="videos" class="grid"></div>
    <div class="bar"><button onclick="toggle('audio')"> Mute</button><button onclick="toggle('video')">
        Camera</button><button onclick="share()"> Screen share</button>
    <label class="btn alt"> Send file<input type="file" hidden onchange="sendFile(this.files[0])"></label></div>
    <h4>Whiteboard <button class="alt" onclick="clearBoard(true)">Clear</button></h4><canvas id="wb" width="800"
        height="450"></canvas></div>
    <div><div id="log"></div><input id="msg" placeholder="Encrypted message... (Enter)"
        onkeydown="if(event.key==='Enter')sendMsg()"></div></div>`;
  addVideo('me', cam, me + ' (you)', true); initBoard();

  socket = io({ auth: { token } });
  socket.on('connect_error', () => alert('Authentication failed - log in again.'));
  socket.on('peers', list => list.forEach(p => { names[p.id] = p.username; call(p.id); })); // newcomer makes the
      offers
  socket.on('peer-joined', p => { names[p.id] = p.username; log('- ' + p.username + ' joined'); });
  socket.on('peer-left', id => { peers[id]?.close(); delete peers[id]; $('#v' + id)?.remove(); log('- ' + (names[id]
      || 'someone') + ' left'); });
  socket.on('signal', m => { chain = chain.then(() => onSignal(m)).catch(console.error); }); // serialise to avoid
      ICE-before-SDP races
  socket.on('chat', async m => { try { log(m.from + ': ' + (await dec(m.p)).text); } catch { log(' Could not decrypt
      a message from ' + m.from + ' (wrong password?)'); } });
```

```
  socket.on('file', async m => {
    try { const f = await dec(m.p); if (!String(f.url).startsWith('data:')) return;
      log(`${esc(m.from)} sent <a download="${esc(f.name)}" href="${f.url}">${esc(f.name)}</a>`, true); } catch { }
  });
  socket.on('draw', async m => { try { seg(await dec(m.p)); } catch { } });
  socket.on('clear', () => clearBoard(false));
  socket.emit('join', room);
}

// ---- WebRTC (mesh: one peer connection per participant) ----
function pc(id) {
  if (peers[id]) return peers[id];
  const c = new RTCPeerConnection(ICE); peers[id] = c;
  cam.getTracks().forEach(t => c.addTrack(t, cam));
  c.onicecandidate = e => e.candidate && socket.emit('signal', { to: id, data: { ice: e.candidate } });
  c.ontrack = e => addVideo(id, e.streams[0], names[id] || 'peer');
  return c;
}
async function call(id) { const c = pc(id); await c.setLocalDescription(await c.createOffer()); socket.emit('signal',
    { to: id, data: { sdp: c.localDescription } }); }
async function onSignal({ from, data }) {
  const c = pc(from);
  if (data.sdp) {
    await c.setRemoteDescription(data.sdp);
    if (data.sdp.type === 'offer') { await c.setLocalDescription(await c.createAnswer()); socket.emit('signal', { to:
        from, data: { sdp: c.localDescription } }); }
  } else if (data.ice) { try { await c.addIceCandidate(data.ice); } catch { } }
}
function addVideo(id, stream, name, muted) {
  let d = $('#v' + id);
  if (!d) { d = document.createElement('div'); d.id = 'v' + id; d.className = 'tile'; d.innerHTML = '<video autoplay
      playsinline></video><span></span>'; $('#videos').append(d); }
  const v = d.querySelector('video'); v.srcObject = stream; v.muted = !!muted; d.querySelector('span').textContent =
      name;
}
const toggle = k => cam.getTracks().filter(t => t.kind === k).forEach(t => t.enabled = !t.enabled);

// ---- Screen sharing: swap the outgoing video track ----
const swap = t => Object.values(peers).forEach(c => { const s = c.getSenders().find(s => s.track && s.track.kind ===
    'video'); s && s.replaceTrack(t); });
async function share() {
  if (sharing) return stopShare();
  try { const s = await navigator.mediaDevices.getDisplayMedia({ video: true }), t = s.getVideoTracks()[0];
    sharing = true; swap(t); addVideo('me', s, me + ' (screen)', true); t.onended = stopShare; } catch { }
}
function stopShare() { sharing = false; const t = cam.getVideoTracks()[0]; if (t) swap(t); addVideo('me', cam, me + '
    (you)', true); }

// ---- Encrypted chat + file sharing ----
async function sendMsg() { const t = $('#msg').value.trim(); if (!t) return; $('#msg').value = ''; log('me: ' + t);
    socket.emit('chat', await enc({ text: t })); }
function sendFile(f) {
  if (!f) return; if (f.size > 5e6) return alert('Max file size is 5 MB');
  const r = new FileReader();
  r.onload = async () => { socket.emit('file', await enc({ name: f.name, url: r.result })); log('You sent ' +
      f.name); };
  r.readAsDataURL(f);
}

// ---- Whiteboard (strokes are encrypted and synced) ----
function seg({ a, b }) { const W = ctx.canvas.width, H = ctx.canvas.height; ctx.beginPath(); ctx.moveTo(a[0] * W,
    a[1] * H); ctx.lineTo(b[0] * W, b[1] * H); ctx.stroke(); }
function clearBoard(local) { ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height); if (local)
    socket.emit('clear'); }
function initBoard() {
  const cv = $('#wb'); ctx = cv.getContext('2d'); ctx.lineCap = 'round'; ctx.lineWidth = 3;
  const pos = e => { const r = cv.getBoundingClientRect(); return [(e.clientX - r.left) / r.width, (e.clientY -
      r.top) / r.height]; };
  cv.onpointerdown = e => { drawing = true; last = pos(e); };
  cv.onpointerup = cv.onpointerleave = () => drawing = false;
  cv.onpointermove = async e => { if (!drawing) return; const p = pos(e), s = { a: last, b: p }; last = p; seg(s);
      socket.emit('draw', await enc(s)); };
}
```

CodeAlpha_RealTimeCommunicationApp/public/common.js

```
const $ = s => document.querySelector(s);
let token = localStorage.token || '', me = localStorage.me || '';
async function api(path, method = 'GET', body) {
  const r = await fetch('/api' + path, { method, headers: { 'Content-Type': 'application/json', Authorization:
      'Bearer ' + token }, body: body ? JSON.stringify(body) : undefined });
  const d = await r.json().catch(() => ({}));
  if (r.status === 401 && token) { localStorage.clear(); location.reload(); }
  if (!r.ok) throw new Error(d.error || 'Something went wrong');
  return d;
}
async function authSubmit(kind) {
  try { const d = await api('/' + kind, 'POST', { username: $('#u').value, password: $('#p').value });
```

```
    token = localStorage.token = d.token; me = localStorage.me = d.username; location.reload();
  } catch (e) { alert(e.message); }
}
function logout() { localStorage.removeItem('token'); localStorage.removeItem('me'); location.reload(); }
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;',
    "'": '&#39;' }[c]));
const authBox = () => `<div class="card narrow"><h3>Login / Register</h3><input id="u" placeholder="username"><input
    id="p" type="password" placeholder="password (6+ chars)"><button onclick="authSubmit('login')">Login</button>
    <button class="alt" onclick="authSubmit('register')">Register</button></div>`;
```

CodeAlpha_RealTimeCommunicationApp/public/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:system-ui,sans-serif;background:#f4f5f7;color:#222}
header{display:flex;justify-content:space-between;align-items:center;padding:12px
    18px;background:#1f2937;color:#fff;position:sticky;top:0;z-index:5}
header a{color:#cbd5e1;margin-left:14px;text-decoration:none;cursor:pointer}header a:hover{color:#fff}
main{max-width:1000px;margin:18px auto;padding:0 12px}
.card{background:#fff;border-radius:10px;padding:14px;margin-bottom:12px;box-shadow:0 1px 3px
    #0002}.narrow{max-width:360px;margin:40px auto}
input,textarea,select{width:100%;padding:9px;margin:5px 0;border:1px solid #cbd5e1;border-radius:6px;font:inherit}
button,.btn{background:#2563eb;color:#fff;border:0;padding:8px
    14px;border-radius:6px;cursor:pointer;font:inherit;display:inline-block}
button.alt,.btn.alt{background:#64748b}button:hover{opacity:.9}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(210px,1fr));gap:12px}.big{font-size:56px;text-align:c
    enter}
.cols{display:grid;grid-template-columns:repeat(3,1fr);gap:12px}@media(max-width:700px){.cols{grid-template-columns:1f
    r}}
.cm{background:#f1f5f9;border-radius:6px;padding:6px 9px;margin:5px 0;font-size:14px}
#toast div{position:fixed;right:14px;bottom:14px;background:#111;color:#fff;padding:10px 14px;border-radius:8px}
```